Abraham Goodman

abrahamgoodman@brandeis.edu

304-319-3670 | 1273 Kings Rd. Morgantown, WV 26508

Senior at Brandeis University majoring in applied mathematics and computer science

All projects made in GameMaker using GameMaker Language. I have also worked with the same partner for all projects.

Little Whammo

2021 - Present

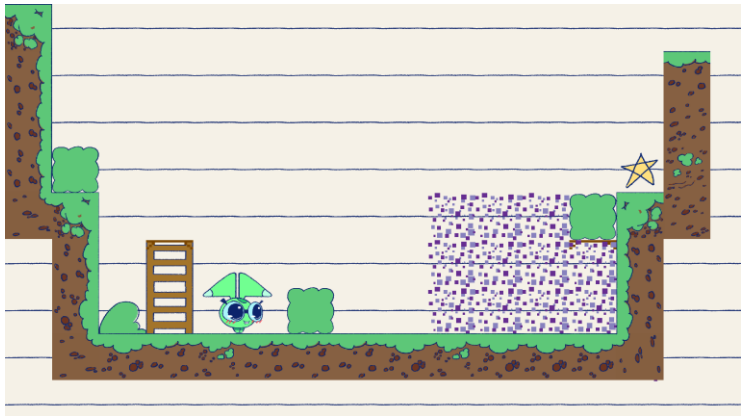
Demo: https://littlewhammo.itch.io/littlewhammo

A puzzle game where you can shoot a pellet to pick up an object, storing it. You can then shoot it out somewhere else and create it in the world. I came up with the concept, and my partner has been helping me throughout the development. We collaboratively design the game, both ideas and levels, and I handle almost all the coding. Our current goals are designing more mechanics and levels, and reworking some of our systems to be easier to add on to in the future.

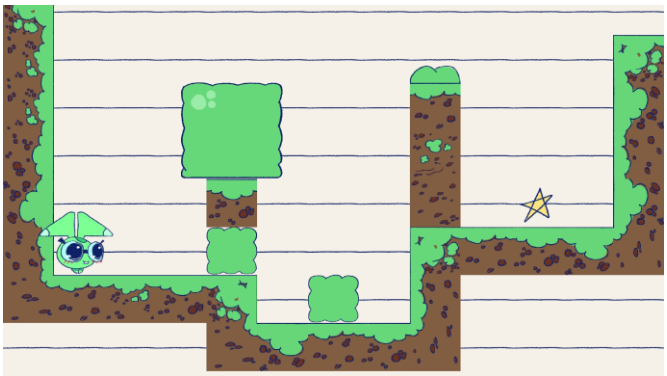
Primary Features:

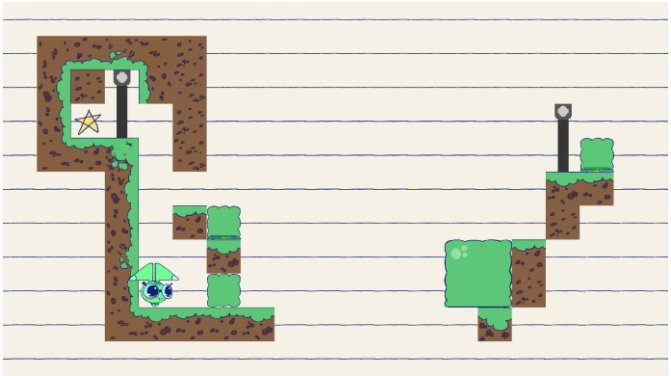
- Inventory system, allowing for storing multiple objects
- Autocorrection when shooting out objects
- Triggers for moving between levels and a hub world
- Many possible player states (climbing, shooting forwards and downwards, stepping up a block given enough space, etc.)
- Unique interactive objects with different mechanics
- Level editor

Highlights:



Video of a full level (click)





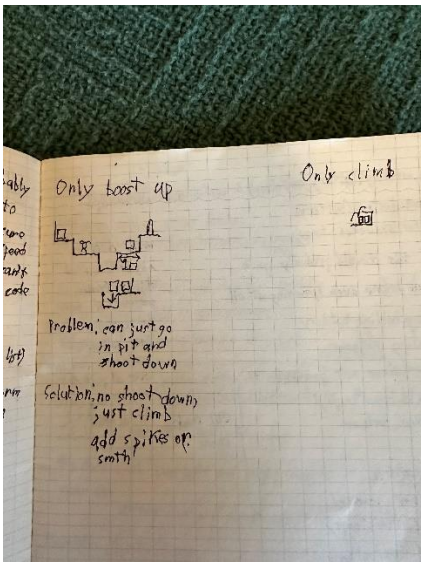
Design Challenges:

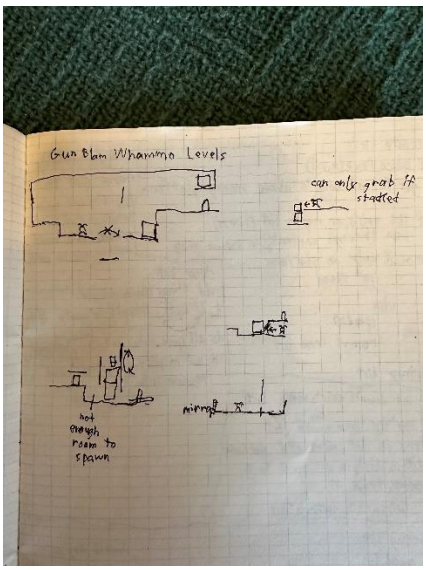
We decided early on that you cannot jump in our game. We thought taking away this basic feature would allow our main mechanic, the ability to pick up and place objects, to shine, and give us many more interesting puzzle scenarios. However, this leads to a host of issues regarding verticality, leading us to two core mechanics: climbing and shooting down. We debated for a while about which of these to add, ending up adding both, and how to implement them in a way that would feel best to the player. Climbing allows the player to ascend short distances, and shooting an object down gives the player a boost upwards.

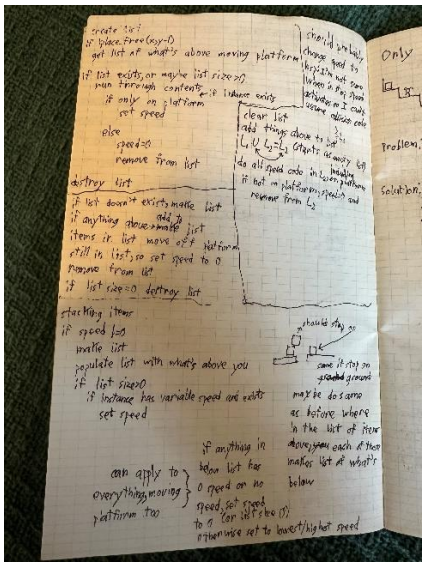
We also talked about moving platforms for a while. As a common videogame feature, we thought we should add it. Coding this with allowing for carrying many objects stacked on top of each other proved to be very difficult, and I tried many ideas.

Ultimately, we thought about if we actually needed moving platforms, and decided they were not necessary. This was after I had done a lot of work trying to implement them, but we thought the game would be better off without them as they added the potential for tight timing, and we wanted our game to be mechanically simple.

Design Screenshots







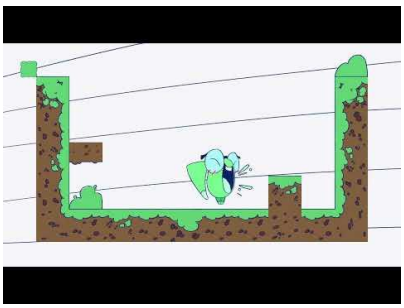
Code Snippets

Both images are part of the code that handles the player inventory. This one distinguishes between the different possible cases of what object is being picked up.

```
//If an object isn't held and the object collided with is holdable, pick it up
if (array_length(obj_player.inventory) < obj_player.inventory_max) {
    if (object_is_ancestor(other.object_index, obj_holdable)) {
        array_push(obj_player.inventory, other.object_index);
        instance_destroy(other);
        exit;
    //If the object collided with is a container, extract a battery
    } else if (other.object_index == obj_container && other.battery_count > 0) {
        array_push(obj_player.inventory, obj_battery);
        other.battery_count -= 1;
        exit;
    }
}

//Shooting a battery into a container
if (array_last(obj_player.inventory) == obj_battery) {
    if (other.object_index == obj_container && other.battery_count < other.max_batteries) {
        array_pop(obj_player.inventory);
        other.battery_count += 1;
        exit;
    }
}
```

This code deals with placing objects when shooting out a pellet. It tries to move an object into a free location to place it, including potentially moving the player.



Video demonstration (click)

```
//The main spawning and placement code
if (array_length(obj_player.inventory) > 0) {
    //Spawns currently held object
    inst = instance_create_layer(x,y,"Instances",array_last(obj_player.inventory));

    var held_item_xoffset = ceil((inst.bbox_right-inst.bbox_left+1)/2);
    var held_item_yoffset = ceil((inst.bbox_bottom-inst.bbox_top+1)/2);
    other_xoffset = ceil((other.bbox_right-other.bbox_left+1)/2);
    other_yoffset = ceil((other.bbox_bottom-other.bbox_top+1)/2);

    //Moves object to the edge of the object it spawned in
    if (direction == 0) {
        inst.x = other.x - other_xoffset - held_item_xoffset;
    } else if (direction == 180) {
        inst.x = other.x + other_xoffset + held_item_xoffset;
    } else if (direction == 270) {
        inst.y = other.y - other_yoffset - held_item_yoffset;
    }

    with (inst) {
        //If it's still stuck in an object
        if (place_meeting(x,y,obj_solid) || place_meeting(x,y,obj_one_way_platform)) {
            inst = instance_place(x,y,obj_one_way_platform);

            if (other.direction == 270) {
                //Moves object to closest free space in a horizontal range
                for (var i = 1; i <= held_item_xoffset; i += 1) {
                    if (!place_meeting(x-i,y,obj_solid)) {
                        x -= i;
                        break;
                    } else if (!place_meeting(x+i,y,obj_solid)) {
                        x += i;
                        break;
                    }
                }
            } else if (inst != noone && y < inst.y) {
                //Spawning above a one-way platform
                for (var i = 1; i <= held_item_yoffset; i += 1) {
                    if (!place_meeting(x,y-i,obj_solid) && !place_meeting(x,y-i,inst)) {
                        y -= i;
```

Monkey Mode

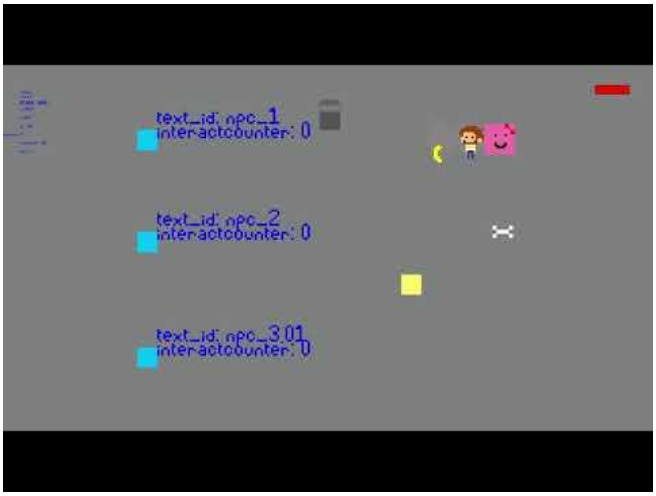
November 2023

We worked on this game for a 2-week game jam, with the theme “Mode”. The player is a monkey transformed into a human, and is going on a date. There is a meter that fills up when monkey-related things happen, like seeing or holding a banana, and the player turns into a monkey when the meter is full. This causes anyone seeing him to panic and the game is lost.

Primary Features:

- Monkey meter that depletes after a brief period of inactivity
- View cones to mimic sight for the player seeing things like bananas or the girlfriend and other NPCs seeing the player turn into a monkey
- Dialogue system with unique dialogue for interacting with something multiple times, and multiple options for the player to choose

Highlights:



Video demonstration (click)

Design Challenges:

We did not end up submitting this game due to the scope of the project we planned being too big and us not having much time to work on it. We will continue to work on the game, and we understand timeframes better for future projects.